□ Week 1 — Solutions

> **Before you read:** Have you genuinely attempted each problem? Solutions are most valuable as a **comparison tool** — look at your own solution first, then compare. Note what you did differently and why the model solution works.

Problem 1 — Personalized Greeting

Original Problem: Write a script that prints a formatted greeting with your name.

Solution

```
# Problem 1: Personalized Greeting
# Prints a formatted welcome message with a decorative border

# Print the top border line
print("============================")

# Print the personalized greeting (replace Alex with your name)
print("  Welcome, Alex!")

# Print the learning statement
print("  I am learning Python.")

# Print the motivational line
print("  Today is a great day to code.")

# Print the bottom border line
print("============================")
```

Expected Output

```
============================
  Welcome, Alex!
  I am learning Python.
  Today is a great day to code.
============================
```

Step-by-Step Explanation

Line	What It Does
`print("==========================")`	Prints 28 = characters as a decorative border
`print("  Welcome, Alex!")`	Prints the greeting — note 2 spaces for indentation
`print("  I am learning Python.")`	Prints the learning statement, also indented
`print("  Today is a great day to code.")`	Prints the motivational message
`print("==========================`	Closes with the same border

Line	What It Does
`===")`	

Alternative Solution (Pythonic)

```
# Using string repetition for the border — more maintainable
border = "=" * 28
print(border)
print("  Welcome, Alex!")
print("  I am learning Python.")
print("  Today is a great day to code.")
print(border)
```

Why the alternative is better: If you want to change the border length, you change one number instead of counting and retyping `=` signs.

Common Mistakes

- **Forgetting the leading spaces** — the original output has 2 spaces before `Welcome`. Match the format exactly.
- **Using `print(= * 28)` without quotes** — the `=` must be a string: `"=" * 28`.
- **Inconsistent border length** — top and bottom borders should be identical.

Key Concepts Demonstrated

- `print()` as the primary output tool
- String literals (text in quotes)
- Code comments using #
- Consistent formatting for readability

Real-World Application

This pattern — header border, content lines, footer border — is used in terminal-based reports, CLI tool output, and log file headers.

Problem 2 — Python vs. Compiled Languages

Original Problem: Write a script explaining the difference between interpreted and compiled languages.

Solution

```
# Problem 2: Python vs. Compiled Languages
# Explains the difference between interpreted and compiled programming

# Print the title
print("=== Python: Interpreted vs. Compiled ===")
```

```
# Print a blank line for spacing
print()

# Explain interpreted languages
print("Interpreted languages execute code line by line at runtime.")

# Explain compiled languages
print("Compiled languages translate ALL code into machine code before running.")

# Print a separator between sections
print("---")

# Explain the key practical difference
print("Python is interpreted: you can run your script immediately after
saving.")

# Print a blank line for spacing
print()

# Summarize the tradeoff
print("Tradeoff: interpreted = faster to develop; compiled = faster to run.")
```

Expected Output

```
=== Python: Interpreted vs. Compiled ===

Interpreted languages execute code line by line at runtime.
Compiled languages translate ALL code into machine code before running.
---
Python is interpreted: you can run your script immediately after saving.

Tradeoff: interpreted = faster to develop; compiled = faster to run.
```

Step-by-Step Explanation

Line	What It Does
`print("=== Python: ... ===")`	Prints a formatted title with decorative border
`print()`	Prints a blank line — no arguments = empty output + newline
`print("Interpreted languages...")`	First explanatory statement
`print("Compiled languages...")`	Contrasting statement
`print("---")`	Visual separator between sections
`print("Python is interpreted...")`	Practical implication for the learner

Alternative Solution (More Advanced)

```
# Using multi-line output structure with separators
title = "=== Python: Interpreted vs. Compiled ==="
separator = "-" * len(title)

print(title)
print()
print("INTERPRETED (Python, JavaScript, Ruby):")
```

```
print("  - Code runs line by line at runtime")
print("  - Errors appear as program runs")
print("  - No compilation step — run immediately")
print()
print("COMPILED (C, C++, Go, Rust):")
print("  - All code translated before execution")
print("  - Errors caught at compile time")
print("  - Faster runtime performance")
print()
print(separator)
print("Python is INTERPRETED — great for automation and rapid development.")
```

Common Mistakes

- **Mixing up comments and print statements** — `# This is a comment` is invisible to the user. `print("This is text")` is visible.
- **No blank lines between sections** — output becomes a wall of text. Use `print()` for spacing.
- **Vague explanations** — be specific. Don't just say "they're different" — say *how* they're different.

Key Concepts Demonstrated

- Using `print()` for structured, readable output
- Comments as documentation for code
- Using blank lines (`print()`) as visual separators

Problem 3 — Comments Practice

Original Problem: Write a commented script displaying string, integer, float, and boolean values.

Solution

```
# Problem 3: Comments Practice
# Demonstrates printing different data types with clear commenting

# Print a heading to identify this script
print("Hello, Python!")

# Print a whole number (integer) — represents a year
print(2024)

# Print a decimal number (float) — represents a price or measurement
print(99.9)

# Print a True/False value (boolean) — represents a logical state
print(False)

# Print an empty line to separate sections visually
print()
```

Expected Output

```
Hello, Python!
2024
99.9
False
```

Step-by-Step Explanation

Line	Data Type	What It Prints
`print("Hello, Python!")`	`str`	The text: `Hello, Python!`
`print(2024)`	`int`	The number: `2024`
`print(99.9)`	`float`	The decimal: `99.9`
`print(False)`	`bool`	The boolean: `False`
`print()`	—	A blank line

Alternative Solution (More Descriptive)

```
# This script demonstrates Python's four main data types

# String (str): text data wrapped in quotes
print("Language: Python")

# Integer (int): a whole number, no decimal point
print(2024)

# Float (float): a decimal number
print(3.14159)

# Boolean (bool): exactly True or False — capitalized!
print(True)

# Blank line for clean output spacing
print()

# Print all four types together using sep to label them
print("str:", "Python", "| int:", 2024, "| float:", 3.14, "| bool:", True)
```

Output of alternative:

```
Language: Python
2024
3.14159
True

str: Python | int: 2024 | float: 3.14 | bool: True
```

Common Mistakes

- **`true` instead of `True`** — Python booleans are capitalized. `true` causes a `NameError`.
- **`false` instead of `False`** — same issue.
- **Putting comments after the code they describe** — conventional style puts comments on

the line *above* the code they explain.

- **Vague comments** — `# print True` is redundant. Better: `# Print whether the system is active (boolean state)`.

Key Concepts Demonstrated

- Four core Python data types: `str`, `int`, `float`, `bool`
- Comment-above-code documentation style
- Using `print()` for blank line spacing
- `print()` with multiple arguments and `sep=`

Real-World Application

In IT automation scripts, you constantly output status information:

```
# Server health check output
print("Server:", "web01")          # str
print("Uptime (days):", 147)       # int
print("CPU Load (%):", 23.7)       # float
print("Healthy:", True)            # bool
```

🏷 Week 1 — Solutions Summary

Problem	Core Concept	Key Takeaway
Problem 1	Formatted output	String repetition `"=" * n` creates cleaner, more maintainable borders
Problem 2	Structured output + comments	`print()` with no arguments = blank line; use this for visual structure
Problem 3	Data types + commenting	Comments go *above* the code they describe; booleans are `True`/`False`

□ Cross References

Previous Topic	Practice Problems
Next Topic	Challenge Lab — ASCII Art
Related Concepts	String variables (Week 2), f-strings (Week 2)
Week README	Week 1 Overview

← *Practice Problems* | *Next: Challenge Lab* →